

RetailPulse

Databricks, PySpark & Spark SQL

Case Study Overview

RetailPulse is a growing omnichannel retail company that receives customer, product and sales-order data from different operational systems. In Part 1, trainees must build the lakehouse and analytical portion of the solution in Databricks Free Edition: load three CSV datasets, clean and transform them with PySpark, store curated results as managed Delta tables, and reproduce important analysis with Spark SQL.

The implementation should remain simple, readable and suitable for classroom demonstration.

Business Scenario

RetailPulse receives data from three operational systems:

- Customer Management System
- Product Management System
- Sales Order System

Source Datasets

Dataset	Rows	Purpose
customers_500.csv	500	Customer profile, segment, location, activity and update history
products_500.csv	500	Product, category, price, cost, supplier, stock and active flag
sales_orders_500.csv	500	Orders, quantities, discounts, status and delivery information

These three datasets will be reused throughout this case study.

Databricks Setup and Bronze Layer

The first stage is to ingest the raw files without performing major business transformations.

Requirements

1. Create a schema named:

```
retail_fresher
```

2. Create a managed volume named:

```
retail_raw
```

3. Upload the following CSV files:

```
customers_500.csv  
products_500.csv  
sales_orders_500.csv
```

4. Read all three CSV files using PySpark.
5. Do not depend on automatic schema inference for the initial raw ingestion.
6. Keep the initial CSV columns as strings in the Bronze layer.
7. Add the following audit columns:

```
source_file  
ingestion_timestamp
```

8. Save the raw data as managed Delta tables.

Required Bronze Tables

```
bronze_customers  
bronze_products  
bronze_sales_orders
```

Data Profiling and Quality Checks

Before cleaning the data, understand what has been loaded. For each DataFrame:

9. Display sample records.
10. Print the schema.
11. Count the total number of rows.
12. Count distinct business keys.
13. Identify duplicate records.
14. Identify missing values.
15. Identify invalid values.

The purpose of this stage is to understand the quality of the source data before applying transformations.

PySpark Data Cleaning and Transformations

Perform the major data-cleaning operations using PySpark.

Customer Transformations

- Trim string columns.
- Standardize upper/lower case where appropriate.
- Replace missing city values with Unknown.
- Convert date and timestamp columns.
- Deduplicate customer records.
- Use `row_number()` to retain the latest customer record based on `updated_at`.

- Remove inactive customers.

Product Transformations

- Trim and standardize string values.
- Convert numeric columns such as price and cost.
- Remove records with invalid prices.
- Remove records with invalid costs.
- Filter inactive products.

Order Transformations

- Convert required numeric and date fields.
- Remove orders where quantity is less than or equal to zero.
- Exclude PENDING and CANCELLED orders from financial analysis.

Required Derived Columns

Create the following calculated columns wherever applicable:

```
gross_amount
discount_amount
net_amount
net_sales
profit_per_unit
delivery_days
late_delivery_flag
order_month
```

These fields should support downstream analytical processing.

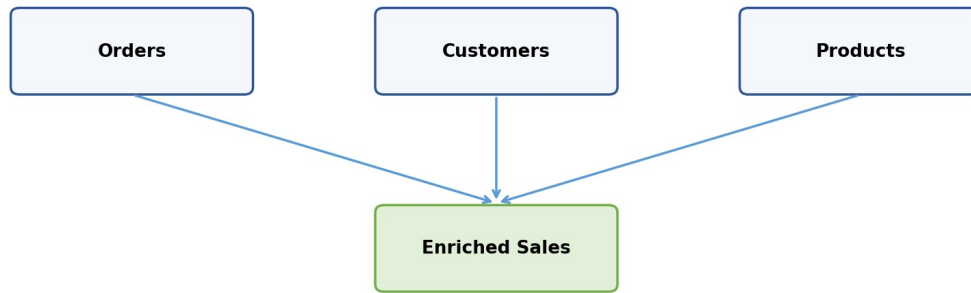
Silver Layer

After cleaning and validating the individual datasets, create the Silver layer. The Silver layer should contain clean and reusable datasets.

- Clean customer data
- Clean product data
- Clean order data
- Enriched sales data

Join the customer, product and order DataFrames to create the enriched sales dataset.

Silver Layer - Enriched Sales Join



Store the cleaned results as managed Delta tables.

Aggregations and Window Functions

1 Monthly Revenue by Category

Calculate revenue grouped by category and order month.

Month	Category	Orders	Quantity	Revenue
2026-01	Electronics	...	...	...
2026-01	Furniture	...	...	...
2026-02	Electronics	...	...	...

2 City-Wise Sales

- Total city sales
- Number of orders
- Average order value

3 Latest Customer Record

Use `row_number()` to retain the most recent customer profile using the latest update timestamp.

4 Latest Order Per Customer

Find the latest order placed by every customer.

5 Product Ranking

Use `rank()` to rank products inside each category based on product revenue.

6 Customer Ranking

Use `dense_rank()` to rank customers within each state.

7 Running Revenue

Calculate the running monthly revenue for each category.

Electronics	
January	100,000
February	140,000
March	120,000
Running Total	
January	100,000
February	240,000
March	360,000

Gold Layer

Create Gold tables containing business-ready analytical results.

Gold Table 1

Monthly Category Sales

Gold Table 2

City Sales

Gold Table 3

Customer Value

Gold Table 4

Top Products by Category

The Gold layer should contain summarized results that can directly support reporting, dashboards or downstream applications.

Spark SQL

After completing the PySpark implementation, reproduce the important business transformations using Spark SQL. Use the managed Delta tables created during the earlier stages.

Spark SQL Requirements

16. Filter completed/valid financial orders.
17. Join the three datasets.
18. Use CAST.
19. Use CASE WHEN.
20. Use date functions.
21. Use string functions.
22. Create a monthly sales summary.
23. Rank products based on category revenue.
24. Find the top three customers in each state.

25. Compare Spark SQL results with equivalent PySpark results.

```
PySpark DataFrame API  
AND  
Spark SQL
```

Practical Assessment Questions 1-17

Data Loading and Understanding

Question 1. Load all three CSV files without automatic schema inference.

Question 2. Print the schema of each DataFrame.

Question 3. Count the total number of rows and distinct business keys.

Question 4. Identify missing and invalid values.

Data Cleaning

Question 5. Deduplicate customers using the latest update timestamp.

Question 6. Convert price, cost, quantity, discount and date fields into appropriate data types.

Question 7. Filter active customers and active valid products.

Question 8. Filter valid financial orders.

Data Transformation

Question 9. Add all required amount and delivery-related columns.

Question 10. Join the customer, product and order DataFrames.

Aggregation and Analytics

Question 11. Find monthly revenue by category.

Question 12. Find city-wise sales and average order value.

Window Functions

Question 13. Find the latest order for each customer.

Question 14. Rank products inside each category.

Question 15. Rank customers inside each state.

Question 16. Calculate running monthly revenue by category.

Spark SQL

Question 17. Reproduce the main analytical outputs using Spark SQL.

Expected Outcome

Bronze Layer

- Three Bronze managed Delta tables

Silver Layer

- Clean customer table
- Clean product table
- Clean order table
- Enriched sales table

Gold Layer

- Monthly category sales
- City sales
- Customer value
- Top products by category

PySpark

- Filters
- Casting
- Joins
- Derived columns
- Aggregations
- Window functions

Spark SQL

- Equivalent SQL implementations of the major business transformations.

Data Quality

- A short validation report comparing input, rejected and processed rows.

Student Submission Checklist

Databricks

- ☐ Databricks notebooks exported as source files
- ☐ Screenshot of Bronze tables
- ☐ Screenshot of Silver tables
- ☐ Screenshot of Gold tables

PySpark

- ☐ PySpark transformation output screenshots

- Aggregation output screenshots
- Window-function output screenshots

Spark SQL

- Spark SQL query screenshots
- SQL result screenshots

Validation

- Data-quality/control-total document

Recommended Submission Folder Structure

```
RetailPulse_Part1/  
├── datasets/  
│   ├── customers_500.csv  
│   ├── products_500.csv  
│   └── sales_orders_500.csv  
├── databricks/  
│   ├── bronze_ingestion.py  
│   ├── silver_transformation.py  
│   ├── gold_analytics.py  
│   └── spark_sql_analysis.sql  
├── screenshots/  
│   ├── bronze/  
│   ├── silver/  
│   ├── gold/  
│   └── spark_sql/  
└── validation/  
    └── data_quality_report.pdf
```

Final Deliverable

```
Raw CSV Data  
↓  
Bronze Layer  
↓  
Data Cleaning  
↓  
Silver Layer  
↓  
Business Transformations  
↓  
Gold Layer  
↓  
PySpark + Spark SQL  
↓  
Validated Analytical Data Pipeline
```

End of Part 1